APZ lab4 report

High-Level Architecture Shift

Previous Architecture: A strictly Synchronous HTTP architecture with static routing. The facade-service communicated directly with the logging-service and counter-service via blocking HTTP requests. If any service went down, the request would fail or hang. Hazelcast was only used as a distributed map for logs.

New Architecture: a hybrid asynchronous architecture with Dynamic Service Discovery. The facade-service now resolves IPs dynamically via a central registry. Communication with the counter-service has been decoupled using an Asynchronous Message Queue (Hazelcast Queue), ensuring that if the database or counter-service goes offline, messages are safely buffered and processed later.

File content explanation in comparison to the previous lab

1. config-server/main.py

IP addresses and URLs for services were hardcoded into the docker-compose.yml and .env variables. This file acts as a Service Registry. It is a lightweight FastAPI application that holds an in-memory dictionary mapping service names to their active IP addresses.

2. facade-service/main.py

Previously: Maintained a static, comma-separated list of logging-service URLs loaded from environment variables and executed a synchronous HTTP POST to the counter-service. If the counter-service was slow or dead, the facade-service would either block or throw an HTTP 500/503 error to the client.

Now: Dynamic Routing. Before making any requests, it queries the config-server to get the latest, active list of URLs for both the logging and counter services.

Message Queue Producer: The synchronous HTTP POST to the counter-service is completely removed. Instead, it connects to the Hazelcast cluster and drops the payload into a distributed messages_queue. The response is immediately returned to the client as "ok", vastly improving response times and fault tolerance.

3. counter-service/main.py

Previously: Acted as a standard REST API. It exposed a POST /message endpoint that synchronously opened a PostgreSQL connection, inserted the message, committed the transaction, and returned a response.

Now: Message Queue Consumer. The POST /message endpoint is gone. Instead on startup it connects to Hazelcast and spawns a background Daemon Thread. This thread continuously listens to the messages_queue. When a message arrives, it pulls it off the queue and writes it to PostgreSQL asynchronously.

Self-Registration: On startup it runs an asynchronous loop to constantly attempt to register its ADVERTISED_URL with the config-server so the facade knows where to send GET requests.

4. logging-service/main.py

Previously: A simple service that received HTTP POSTs, saved them to a Hazelcast Distributed Map, and returned them on a GET request. It was entirely oblivious to the rest of the network.

Now: The core logging logic remains the same (still using the Hazelcast Map), but it now includes a Self-Registration mechanism. When the container starts, an asynchronous starts and pings the config-server with its SERVICE_NAME and ADVERTISED_URL so the facade can discover it dynamically for load balancing.

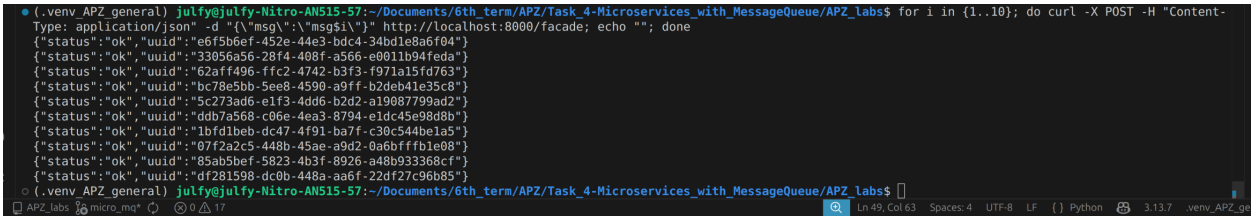
5. docker-compose.yml

Previously: Defined the static LOGGING_NODES and MESSAGES_URL for the facade-service. The architecture was rigid; adding a fourth logging service required changing the facade's configuration and restarting it.

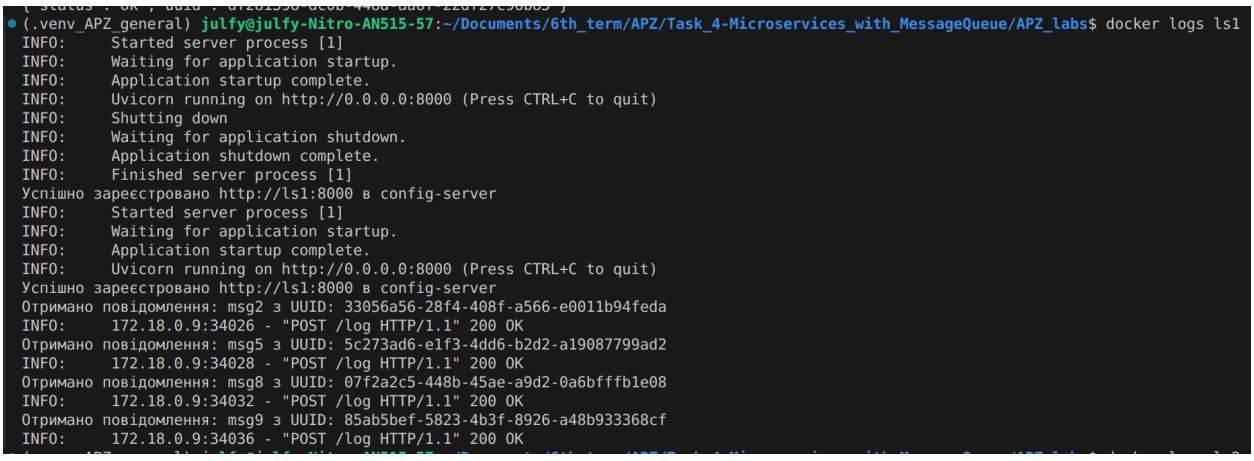
Now: Added the new config-server container. Injected new environment variables (CONFIG_SERVER, SERVICE_NAME, ADVERTISED_URL) into every microservice.

The architecture is now highly scalable. If you add logging-service-4 to the compose file, it will automatically register itself with the config-server, and the facade-service will begin sending traffic to it without any restarts or manual configuration updates.

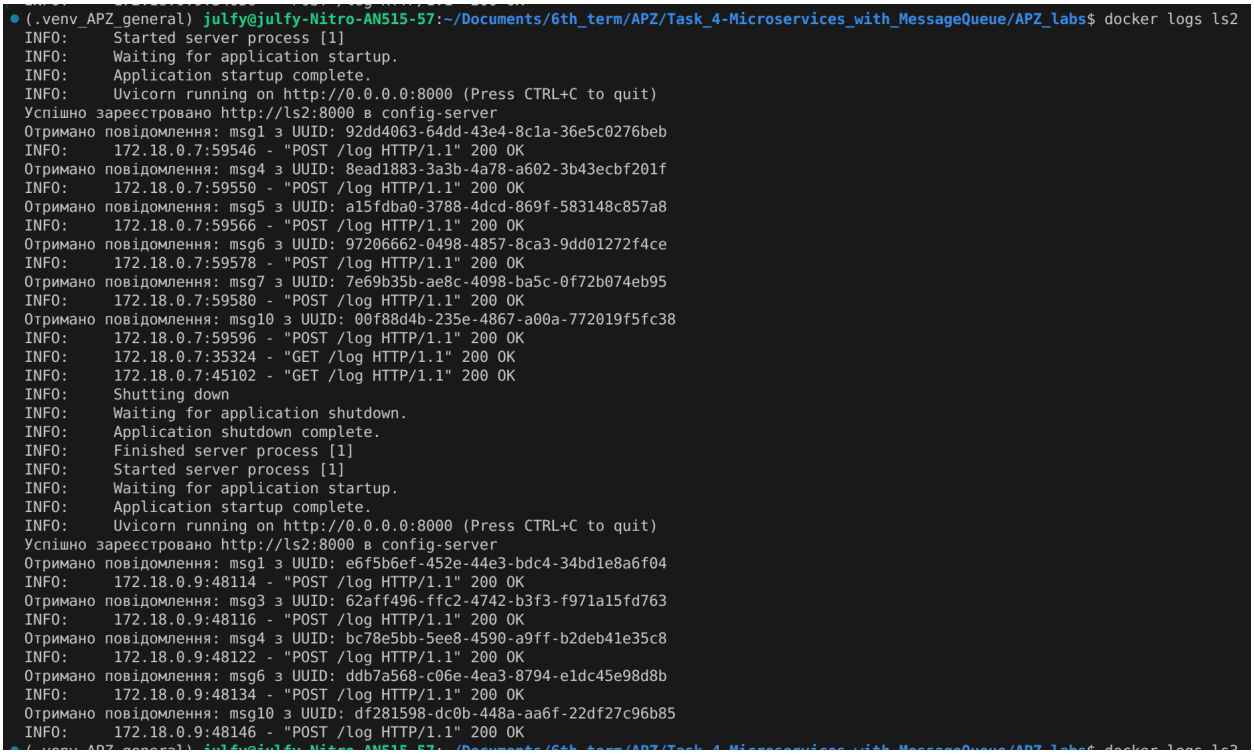
Executed a bash loop sending 10 POST requests (msg1–msg10) to `facade-service` on port 8000. Each response returns `{"status":"ok", "uuid":"..."}` immediately proving the facade does not block on counter-service writes and acts purely as a message queue producer



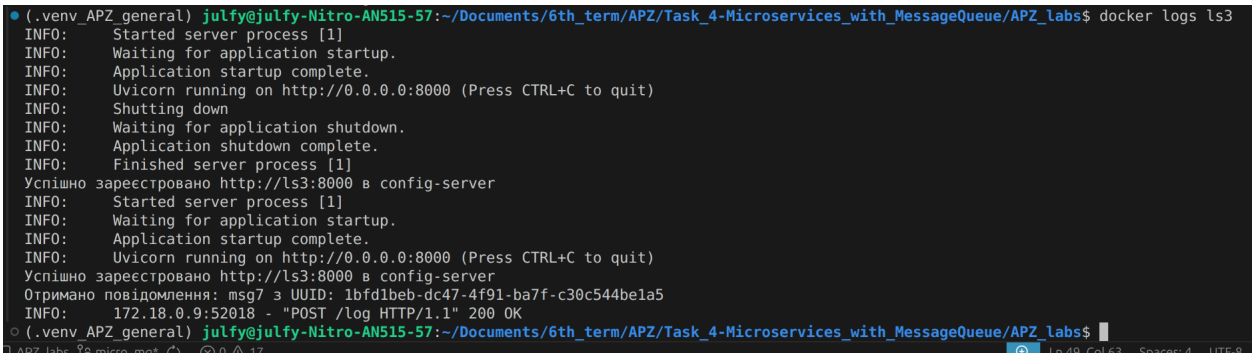
Logging-service instance #1. Shows successful self-registration with config-server ("Успішно зареєстровано http://ls1:8000 в config-server"), then reception of a subset of messages (msg2, msg5, msg8, msg9). 200 OK responses on POST /log confirm Hazelcast Map writes



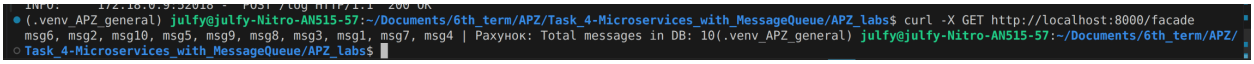
Logging-service instance #2 received a different subset (msg1, msg3, msg4, msg5, msg6, msg7, msg8, msg10) plus GET /log requests. This proves facade-service is round-robin / random-selecting targets from the dynamic list returned by config-server



Logging-service instance #3 received msg7. Together with ls1 and ls2 logs, this demonstrates the requirement "messages are received by different logging-service instances" messages are replicated via Hazelcast Map but each POST lands on exactly one instance chosen dynamically

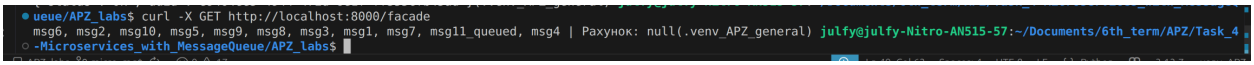


GET returns the aggregated message list from any logging-service (read via Hazelcast Map values) plus "Рахунок: Total messages in DB: 10" from counter-service. Confirms both read paths work and the PostgreSQL row count matches the 10 POSTs

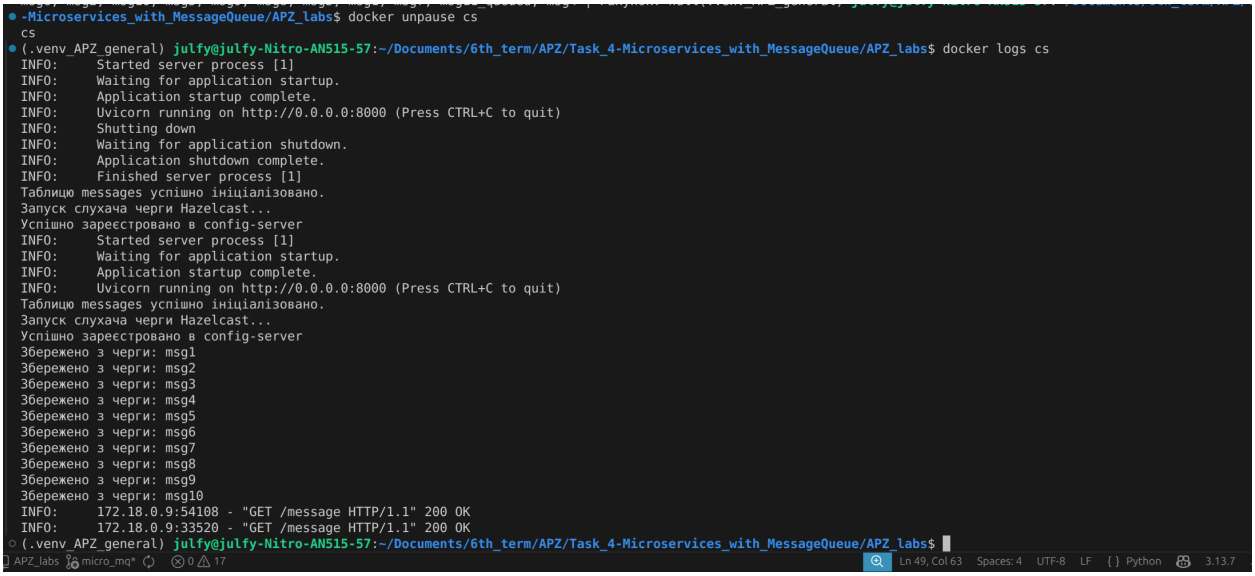


Counter-service is frozen via `docker pause`. A new POST (msg11_queued) still returns `{"status":"ok"}` instantly — proving fault tolerance: the facade only drops the payload into Hazelcast Queue and does not depend on counter-service availability

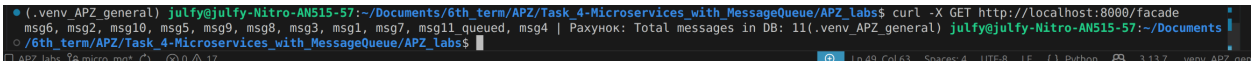
GET returns the message list (still served by logging-service) but "Рахунок: null" because counter-service cannot respond. Exactly matches the spec: "GET-запити ... мають повертати null у якості значень на рахунку"



After unpause, counter-service consumer thread drains the Hazelcast Queue ("Збережено з черги: msgN" lines). Demonstrates automatic catch-up: buffered messages are persisted to PostgreSQL without any manual retrigger



"Total messages in DB: 11" — msg11_queued has been processed from the queue after recovery. Closes the fault-tolerance loop: zero messages lost during the outage



https://github.com/Bohdanok/APZ_labs/tree/micro_mq